

GENERATIVE AI AS A TOOL FOR TRANSFORMING DIGITAL INFORMATION INTO 3D GEOMETRY

Abstract: The paper analyzes the use of generative artificial intelligence for the automatic generation of three-dimensional QR code models for additive manufacturing. The study starts from the limitations encountered in the direct generation of 3D files through natural language instructions, such as inconsistency of results, topological errors and incompatibilities with slicing software used in multi-material 3D printing. Following an iterative process of refining the prompts and analyzing the errors, a universal Python code is developed, capable of reproducibly generating correct 3D QR code models by materials. The results show a significant increase in the success rate, from approximately 40–50% in the initial phase to over 90% after optimization, with fully functional and scannable models.

Key words: generative artificial intelligence, 3D modeling, 3D printing, graphic communication.

1. INTRODUCTION

One of the more recent avenues of exploration in the field of artificial intelligence is the application of generative artificial intelligence to create 3D models and CAD files automatically. These models enable the direct translation of code written in natural languages into code that can execute the object definition in a computer program, such as Python or OpenSCAD, to create a fully defined digital object [1]. In the context of 3D QR code development, the benefits of artificial intelligence are quite clear: they can save time, require no knowledge of 3D modeling, and enable quicker integration with AM systems [2, 3].

The primary concern with artificial intelligence, however, is the unpredictability of its answers and the lack of consistency in the generated files. Initial experiments with artificial intelligence have revealed that, although it can generate .obj files that can be used with slicing programs like Bambu Studio, there is still considerable scope for improvement, with errors in topology, material separation, and geometry affecting the functionality of the generated 3D QR code.

The goal of this paper is to transcend these limitations by determining the capabilities of artificial intelligence and developing a universal, valid, and customizable Python code that can produce accurate, scalable, and fully compatible 3D QR code models for multi-material AM processes [4]. This can be achieved by using the ChatGPT model and its application of a rigorous methodological process of testing, optimizing, and validating the 3D QR code model for its reproducibility and application for rapid prototyping or manufacturing purposes[5].

2. LITERATURE REVIEW

The QR (Quick Response) codes are defined by the international standard ISO/IEC 18004, which defines the data structure, the encoding mechanism, and the minimum size of the QR code that can be scanned automatically [6]. The QR codes are two-dimensional

codes consisting of a matrix of black and white modules arranged in a square pattern along with positioning markers and quiet zones. When converting these codes into 3D models for the purposes of additive manufacturing, the proportions of the model must be accurately represented while maintaining sufficient contrast between the different parts of the QR code so that the model can be scanned optically. The minimum module size, flatness of the surface, and the accuracy of the printing are critical factors since the mechanical differences that are introduced during the process of additive manufacturing can hinder the readability of the QR code. Research has indicated that for optimal scannability of the QR code during the FDM process, the ratio of the module size to the resolution of the printing process must be at least 4:1, affecting the scaling of the model.

Artificial intelligence and large language models (LLMs) like ChatGPT have started to be investigated as means to automatically generate source code in programming languages like those used in computer-aided design [7, 8]. Such code can be Python or OpenSCAD code that may generate parametric designs, including complex designs like 3D QR codes [9]. The most important benefit of using these technologies is the ease with which the user may describe the object he or she wants to design. The AI will then generate code to automatically design the CAD representation of the desired object. The literature states that these technologies may prove to be useful in shortening design time and making CAD more accessible [10].

However, there are certain challenges associated with the use of AI in generating 3D models directly. Firstly, there is a possibility that the generated models may be inconsistent, with syntactic errors in the code or topologically invalid geometries, thus making it difficult to use language models in generating 3D models directly [11, 12]. Secondly, it is found that not all generated models are compliant with strict material separation requirements or are optimized for file formats supported by slicing tools like Bambu Studio or PrusaSlicer [13]. Furthermore, there is no mechanism within the AI to

validate whether the generated geometric model is functional or not, or whether it is possible to scan the QR code once it is printed [14]. All these challenges have created a necessity to have an iterative process, like the one proposed in this paper, that can provide reproducible results with the help of human expertise, along with the capabilities of AI [15].

3. METHODOLOGY

3.1. Experimental protocol

To test the reliability of ChatGPT in the automatic generation of three-dimensional models of QR codes compatible with the Bambu Studio slicing tool, a specific experimental design was developed, focusing on assessing the technical reliability of the generated files, as well as on the potential limitations of the process. The experimental design was carried out in a controlled environment, using a stable version of the ChatGPT 5.1 model, with default response parameters and without imposing any restrictions on the accessibility of the code generation tool. This design was used to ensure the reproducibility of the experimental results, avoiding potential uncontrolled variations that could have affected the interpretation of the data.

To carry out the testing, a series of consecutive sessions were designed, during which the chatbot was requested to directly generate a downloadable file in .OBJ format, compatible with the Bambu Studio tool, which could be imported without errors and render a functional QR code. During each session, the basic structure of the request, as well as the parameters of the chatbot, were maintained constant, with only specific details of the instructions being changed to assess their influence on the quality of the generated file. This design allowed for the assessment of the chatbot's success rate, as well as the potential sensitivity of the model to different formulations of the request.

Each generated file has been validated by following a three-step validation process consisting of geometric integrity verification, QR code scannability validation, and compatibility with the Bambu Studio workflow. The validation process has been carried out according to the methodology proposed by Mititelu et al. [16]. The geometric integrity of generated files has been verified by visual inspection in CAD environments. The files' scannability has been validated by employing QR code reader applications under normal lighting conditions. The compatibility with the Bambu Studio workflow has been verified by directly importing the file into the software and simulating the slicing process to exclude any potential incompatibilities or issues with interpreting the multi-material geometry.

To ensure full reproducibility of the proposed workflow, the complete Python implementation for generating multimaterial 3D-printable QR codes is publicly available at: <https://github.com/razvanmt-byte/QR-code-generator>. The repository contains the source code, usage instructions, and example output files in .OBJ format. Detailed configuration files for slicing (Bambu Studio and PrusaSlicer) are also provided to replicate the experimental setup described in this study.

3.2. Optimization process

The initial results confirmed that although it is possible to produce a 3D file in .OBJ format containing a functional QR code with ChatGPT, the consistency of the results varies. In most cases, the produced files had structural defects ranging from topology issues to the impossibility of importing them into Bambu Studio. Figures 1 and 2 are illustrations of such issues. These limitations required an optimization process to increase the success rate of the desired outcome to produce a robust Python code. The research by Ermolai et al. [17] points to the fact that dimensional accuracy in Fused Filament Fabrication in Additive Manufacturing processes depends on a complex combination of factors. These factors include the kinematic precision of the device used in the process, the thermoviscoelastic properties of polymeric materials used in Fused Filament Fabrication, and variations in extrusion processes. Therefore, to be able to scan a QR code produced by a 3D printer, it must be optimized to be unaffected by all the disturbances or errors mentioned above [18].

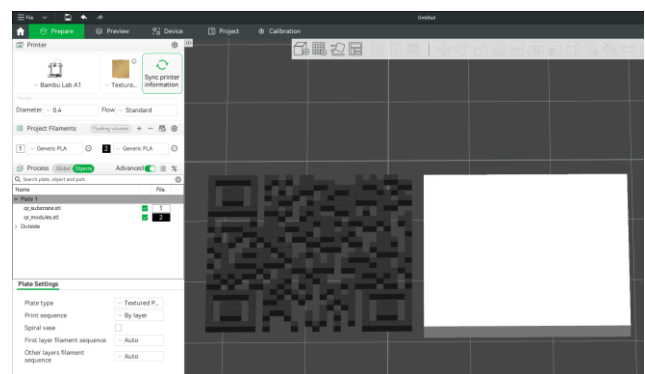


Figure 1 Model imported into Bmbuu Studio program

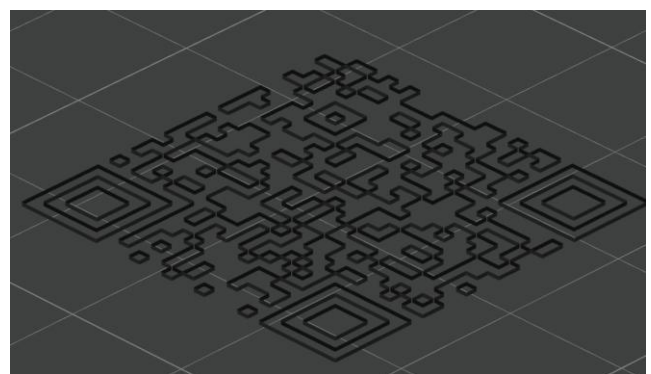


Figure 2 Model generation errors

The first step in this optimization process was to analyze the defective files to understand patterns in errors. It was found that a significant number of errors arose from the fact that the model did not completely understand the requirement, resulting in missing essential steps in the coding process or generating geometric structures that were not aligned or overlapped inappropriately. In other cases, although the code was generated correctly, execution of that code resulted in models with disproportionate dimensions or incorrect

color mapping between the black and white components of the QR code.

Therefore, on the basis of this analysis, requirements for ChatGPT were refined, with emphasis on the instructions provided, geometric parameters, and inclusion of a requirement for a distinct multi-material model with appropriate layering for QR modules.

3.3. Steps used to print the generated model

The practical validation process of the optimized Python code included downloading and importing the 3D file generated by ChatGPT into the Bambu Studio slicing software, followed by configuring the printing parameters and visually checking the model.

3.3.1. Downloading the AI-generated file

After executing the universal Python code, ChatGPT produced an .OBJ file with two different objects: the motherboard, represented in white, and the QR code modules, represented in black. The file was saved locally, retaining its multi-material composition, which is essential for successful printing.

This file format enables Bambu Studio to automatically identify different objects, facilitating the application of different colors and materials.

3.3.2. Importing the file into Bambu Studio

The file in OBJ format was loaded into the Bambu Studio program. The program recognized that two geometric entities were defined within the file, allowing the user to choose between two import options: "Separate meshes" or "Objects as different extruders." This ensures that the two colors of the QR code are correctly assigned to two different extruders, thereby retaining the contrast between them (see figure 3).

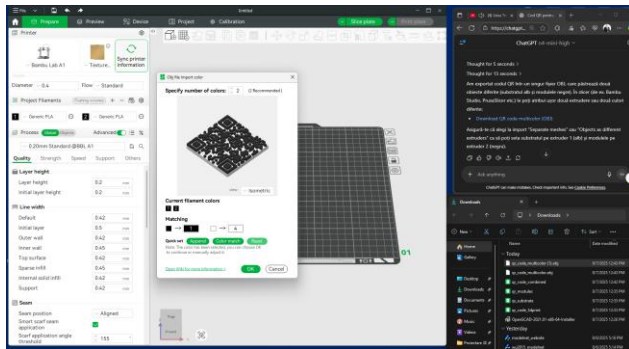


Figure 3 Model imported into Bambu Studio program

3.3.3 Visual verification of the result

Once this was done, the model was displayed correctly in the software's 3D viewer, with the dark modules standing out against the white background. The final image, as seen in Figure 4, demonstrates that the proportion and shape of the QR code have been maintained, while the contrast is also optimal for scanning. This step, checking the model visually in the slicing environment, is a vital step before the actual printing, as it allows you to verify that there is a match between the digital model and the physical object.

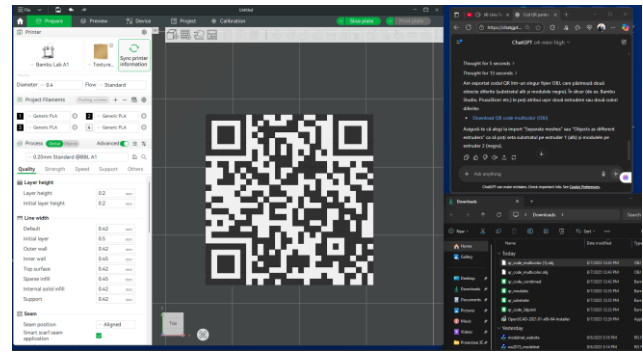


Figure 4 The final model after applying feedback and generating changes by AI

There were times, though, that the files created by the AI could not be automatically differentiated as separate components of a single object. To address this, a tweak had to be made by manually selecting the object, then opting for the "split" option from the menu of the slicer to manually separate the components of the 3D object. Ultimately, the first part was colored white, as shown in Figure 5.

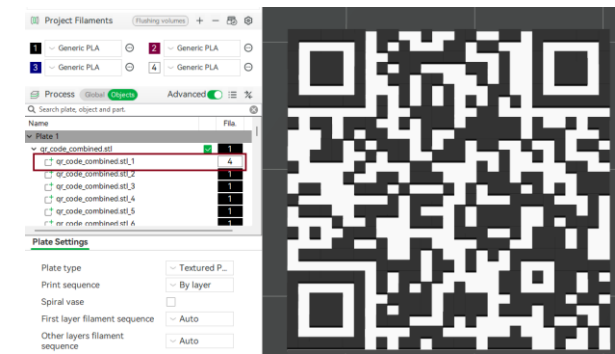


Figure 5 Selecting colors for each layer

3.4. Final Python code and customization instructions

The final step of the methodological process brought everything together into one solid and tested version of the Python code. It should be understood that this isn't merely the direct copy of the code generated by ChatGPT before. Rather, it is an optimized distillation of the entire process. The code is designed to meet Bambu Studio requirements strictly, maintain the distinction between materials (white base and black modules), and preserve the proportion of geometric scaling regardless of the density of the QR matrix.

From an architectural point of view, the script implements five main stages:

– Binary matrix generation

The qrcode library is used to generate the binary matrix corresponding to the QR code associated with a digital link. In this case, the link is https://linktr.ee/Mititelu_Razvan, which will be converted into a matrix of binary points (1 – black module, 0 – empty space). The resulting matrix is transformed into a numpy array of type int, to allow for further numerical processing.

– Defining dimensional parameters

The physical dimensions of the board and QR modules are defined in millimeters. The base board has a fixed size of 25x25 mm and a thickness of 2 mm, while the QR modules have a thickness of 1 mm. The side dimension of each module is calculated proportionally based on the resolution of the binary matrix.

– Creating the motherboard

A rectangular parallelepiped (box) is created to serve as a substrate. It is translated so that its center is located in the middle of the three-dimensional volume and is colored white using the RGBA code [255, 255, 255, 255].

– Creating black modules

Each element in the QR matrix is traversed, and if the value is 1 (active module), a cuboid with dimensions $\text{module_size} \times \text{module_size} \times \text{module_thickness}$ is generated, positioned correctly above the base plate. Spatial translation ensures the correct alignment of each module in 3D coordinates, and the color black is explicitly set.

– Combining the modules that make up the QR code

All black modules are combined into a single mesh object using `trimesh.util.concatenate`. Next, a 3D scene is created that includes two independent geometric entities: the white substrate (White_Base) and the black QR code (Black_QR_Modules). This separation is essential for multi-material export and compatibility with advanced 3D printers (e.g., Prusa, Bambu), which can detect and treat the two objects separately.

– Export

The model is exported to an .OBJ file with two distinct objects, ready to be imported into pre-processing applications for 3D printing or CAD visualization. The OBJ format preserves the topology of objects and visual metadata (colors) and is widely accepted in the 3D modeling ecosystem.

The final code, adaptable for any other generation of a particular QR code, is as follows:

```
import qrcode
import numpy as np
import trimesh
from trimesh.creation import box
from trimesh.scene import Scene
# Generate QR code image
qr =
qrcode.QRCode(error_correction=qrcode.constants.ERROR_CORRECT_H)
qr.add_data('https://linktr.ee/Mititelu_Razvan')
qr.make()
matrix = np.array(qr.get_matrix(), dtype=int)
# Constants
qr_size = 25.0 # mm
base_thickness = 2.0 # mm
module_thickness = 1.0 # mm
module_size = qr_size / matrix.shape[0] # mm per module
# Create the white base plate
base_plate = box(extents=[qr_size, qr_size, base_thickness])
base_plate.apply_translation([qr_size / 2, qr_size / 2, base_thickness / 2])
```

```
base_plate.visual.face_colors = [255, 255, 255, 255] # White
# Create black QR code modules
black_modules = []
rows, cols = matrix.shape
for y in range(rows):
    for x in range(cols):
        if matrix[y, x] == 1:
            block = box(extents=[module_size, module_size, module_thickness])
            block.apply_translation([
                (x + 0.5) * module_size,
                (rows - y - 0.5) * module_size,
                base_thickness + module_thickness / 2
            ])
            block.visual.face_colors = [0, 0, 0, 255] # Black
            black_modules.append(block)
# Combine black modules into one mesh
qr_code_mesh =
trimesh.util.concatenate(black_modules)
# Create scene with two separate geometries
scene = Scene()
scene.add_geometry(base_plate,
node_name='White_Base')
scene.add_geometry(qr_code_mesh,
node_name='Black_QR_Modules')
# Export as a single OBJ file with two objects
output_path = "/mnt/data/qr_code_plate.obj"
scene.export(output_path)
output_path
Result
'/mnt/data/qr_code_plate.obj'
```

Customizing the code is extremely simple and accessible even to users without advanced programming experience. The only element that needs to be modified to generate a custom QR code is the URL entered in the line:

```
python
qr.add_data('https://your_link.com')
```

By replacing this address with your own link, the script will instantly produce a 3D model of a QR code that encodes the respective information. The physical dimensions of the model can be adjusted by changing the `qr_size` value (in millimeters), and the thickness of the elements can be adapted according to the mechanical or aesthetic requirements of the application.

4. RESULTS

The experimental protocol, as well as the optimization process, significantly improved the quality of the 3D files produced. In the early experiments, when ChatGPT was provided with general requirements, the overall rate of fully functioning .OBJ files that could be successfully imported into Bambu Studio and scanned was merely 40-50%. The problems ranged from topology issues, lack of unique materials, to incorrect QR module alignment, all of which negatively impacted scanability as well as compatibility with multi-material printing.

The final version of the code proved to be reliable in terms of compatibility with Bambu Studio, as all 3D files produced with the new version of the code were

successfully imported into the platform without any problems, allowing for an accurate preview of the slicing process with materials separated for multi-material printing.

Following this polished prompt and locking in a universal Python version, the success rate improved significantly, achieving rates above 90% on a regular basis. This is due to the well-defined geometric parameters, logical order of modeling, and utilization of validated Python libraries at all stages. Additionally, defining the requirement to separate materials in the final file has also removed instances where the model resulted in a single piece, which had limited functionality in Bambu Studio.

The visual differences are also evident. The final version has the black modules resting on the base plate with millimeter-level precision to match the proportions of the QR matrix. However, this has functional implications beyond aesthetics. The QR code reader must read it correctly. Scanning tests on a physical model produced with a 3D printer resulted in a 100% recognition rate under normal lighting conditions. The algorithm also optimized the QR code's shape to enable it to be printed on any 3D printer. Figure 6 presents a comparison of the pre-optimized QR code with one produced with the automatic optimization by the AI algorithm.

This achievement is important not only from the end user's perspective, but also from the perspective of integrating this flow into an automated industrial process, where reliability is essential for reducing production times and costs.

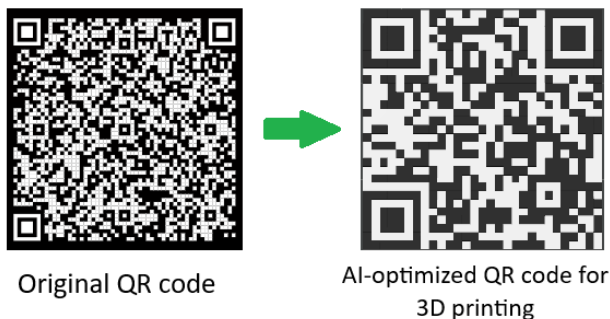


Figure 6 Visual comparison of the optimization achieved

The results demonstrate that, with iterative refinement of prompts and generalization of instructions into general Python code, it is possible to convert ChatGPT from a probabilistic pattern generator into a predictable, efficient solution for generating 3D QR codes that can be directly integrated into additive manufacturing workflows, not only for custom applications but also for standardized methods for integrating AI into CAD design for 3D printing.

5. CONCLUSIONS

This study demonstrates that, not only can generative artificial intelligence, such as the ChatGPT 5.1 model, be integrated into CAD design for additive manufacturing,

but it can also be effective with the right support of a rigorous testing and optimization process. The results clearly demonstrate that, without clear instructions, artificial intelligence can create inconsistent, non-functional results, which limits its overall usefulness in industrial settings. However, by using an iterative process to improve the requirements and analyze the errors generated by the artificial intelligence, we have developed a universal Python code that can create 3D models of QR codes consistently, which can be used with the Bambu Studio platform with a 100% success rate under normal circumstances.

The process used to develop the code has several important benefits to the overall field of additive manufacturing. First, it can be easily customized to create unique QR codes by modifying the link used in the code, which does not require advanced programming knowledge. Second, the final code in .OBJ format with separate color entities can be used to create multimaterial prints, which is an important factor in settings where readability is important. Third, the high process reliability ensures that the overall process can be optimized to prevent the rejection of parts due to coding errors or software incompatibilities.

Other than QR codes, the approach that has been outlined can also be extended to other identifiers that are embedded on 3D parts, such as 3D barcodes or other visual identifiers that are used for authenticity purposes. This approach can also be integrated into IoT platforms that can automate the entire customization and manufacturing process, thus allowing the creation of custom parts that are uniquely identified from a centralized database.

In the future, the approach can also be further improved using advanced prompt engineering techniques and the integration of other AI models that are specialized for 3D geometry. Testing the approach using a wider range of slicing software and printers of different architectures can also be performed to further validate the universality of the approach.

The Python script provided is an automated script that can be used for the creation of three-dimensional QR codes that can be used for additive manufacturing purposes. Using the `qrcode`, `numpy`, and `trimesh` libraries, the script can create a functional model that can be exported for use with multiple materials or custom industrial purposes. The method can be extended for insertion into housings, mechatronic components, or smart tags within an Industry 5.1 architecture.

REFERENCES

- [1] Daareyni, A., A. Martikkala, H. Mokhtarian, and I. F. Ituarte, (2025), Generative AI meets CAD: enhancing engineering design to manufacturing processes with large language models, *The International Journal of Advanced Manufacturing Technology*, doi: 10.1007/s00170-025-15830-2.
- [2] Zhang, L., B. Le, N. Akhtar, S.-K. Lam, and D. Ngo, (2026), Large Language Models for Computer-Aided Design: A Survey, *ACM Comput. Surv.*, vol. 58, no. 9, p. Article 225, Art no., doi: 10.1145/3787499.

- [3] Dori, D. and K. Tombre, (1995), From engineering drawings to 3D CAD models: are we ready now?, *Computer-Aided Design*, vol. 27, no. 4, pp. 243–254, Art no. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0010448595911347>.
- [4] Jakub, K., M. Aleksandra, M. Dariusz, and Rojek, (2024), Enhancing 3D Printing with Procedural Generation and STL Formatting Using Python, *Applied Sciences*, vol. 14, no. 16, p. 7299, Art no. [Online]. Available: <https://www.mdpi.com/2076-3417/14/16/7299>.
- [5] Peckham, O. *et al.*, (2025), Artificial Intelligence in Generative Design: A Structured Review of Trends and Opportunities in Techniques and Applications, *Designs*, vol. 9, no. 4, p. 79, Art no. [Online]. Available: <https://www.mdpi.com/2411-9660/9/4/79>.
- [6] *ISO/IEC 18004:2015 Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*, International Organization for, S., Geneva, Switzerland, 2015.
- [7] Mititelu, N.-R. and *et al.*, (2025), Use of AI Language Models in Generating 3D Printable Models, *Acta Technica Napocensis - Series: Applied Mathematics, Mechanics, and Engineering*, vol. 68, no. 1S. [Online]. Available: <https://atna-mam.utcluj.ro/index.php/Acta/article/view/2779>.
- [8] Shafiq, O., A. Rahmat, A. Alexiadis, and B. Ghiassi, "Evaluating the Performance of Large Language Models for Geometry and Simulation File Generation in Physics-Based Simulations," *Applied Sciences*, vol. 15, no. 22, p. 12114doi: 10.3390/app152212114.
- [9] Guan, Y., X. Wang, X. Xing, J. Zhang, D. Xu, and Q. Yu, (2025), CAD-Coder: Text-to-CAD Generation with Chain-of-Thought and Geometric Reward, *arXiv preprint arXiv:2505.19713*.
- [10] Picard, C. *et al.*, (2025), From concept to manufacturing: evaluating vision-language models for engineering design, *Artificial Intelligence Review*, vol. 58, no. 9, p. 288, Art no., doi: 10.1007/s10462-025-11290-y.
- [11] Mititelu, N. R., (2024), Theoretical aspects regarding AI implementation in optimized STL file generation, *International journal of engineering research & technology (IJERT)*, vol. 13.
- [12] Deng, H., S. Khan, and J. A. Erkoyuncu, (2024), An Investigation on Utilizing Large Language Model for Industrial Computer-Aided Design Automation, *Procedia CIRP*, vol. 128, pp. 221-226, Art no., doi: <https://doi.org/10.1016/j.procir.2024.07.049>.
- [13] Mititelu N. R, Hobjala R., and P. C., (2024), THE USE OF ARTIFICIAL INTELLIGENCE FOR DETECTING AND SOLVING PRINTING ISSUES *TEHNOMUS - New Technologies and Products in Machine Manufacturing Technologies* vol. 31.
- [14] Haque, S., M. H. Tanveer, and R. C. Voicu, (2025), "AI-Driven 3D Printing: Generative Gcode for Quality and Efficiency," in *SoutheastCon 2025*, 22-30 March 2025 2025, pp. 1282-1287, doi: 10.1109/SoutheastCon56624.2025.10971689.
- [15] Paviot, T., V. Fortineau, and S. Lamouri, (2025), Iterative Diagnosis-Driven Augmented Generation (IDDAG) for Programmatic 3D CAD. [Online]. Available: <https://www.scitepress.org/Papers/2025/134435/134435.pdf>.
- [16] Mititelu, N.-R., M. Nicolau, A. I. Irimia, and C.-G. Gradinariu, (2026), Methodological Framework for Evaluating the Generative Capabilities of AI Assisted CAD Parametric Modeling for AM, *Materials Research Proceedings*, vol. 61, pp. 77-83, Art no.
- [17] Ermolai, V. *et al.*, (2024), XYZ Calibration Cube – A Misleading Tool for Achieving Print Accuracy, *Materials Research Proceedings*, vol. 46, pp. 23-34, Art no.
- [18] Fudos, I. *et al.*, (2020), A characterization of 3D Printability, *arXiv preprint arXiv:2010.12930*.

Author(s):

PHD student Eng. Cosmin Gabriel GRĂDINARU, "Gheorghe Asachi" Technical University of Iași, Department of Machine Manufacturing Technology, cosmin-gabriel.gradinaru@student.tuiasi.ro

Teaching Assistant Eng. Nicolae-Razvan MITITELU, "Gheorghe Asachi" Technical University of Iași, Department of Engineering Graphics, nicolae-razvan.mititelu@academic.tuiasi.ro

PhD student Eng. Oussama ABDA Affiliation, University of Djelfa, Algeria, oussama.abda@univ-djelfa.dz

Lecturer PhD. Eng. Irina IGNĂTESCU-MANEA, "Gheorghe Asachi" Technical University of Iași, Department of Engineering Graphics, irina.ignatescu-manea@academic.tuiasi.ro

Assoc. Prof. Ph.D. Eng. Liviu PRUNA, Director of Department, "Gheorghe Asachi" Technical University of Iași, Department of Engineering Graphics, liviu.pruna@tuiasi.ro